MOJO-V EXO LIBRARY: PURPOSE, DESIGN, WORKFLOW, AND LLVM PORT TRADE-OFFS

1) WHAT THE EXO LIBRARY IS

The **Mojo-V EXO library** is a C/C++ header-based compatibility layer that exposes encrypted integer and floating-point programming abstractions to software today, on top of existing compilers, while targeting the Mojo-V execution model.

At a code level, EXO is centered on:

- ‘exo/mojov-exo.h‘: C++ encrypted scalar wrappers (‘inte_t<Bits, Signed>‘, ‘fpe_t<Bits>‘) and overloads for arithmetic, bitwise, relational, conversion, and ‘cmov‘ operations.
- ‘exo/mojov-intrinsics.h‘: low-level inline-assembly intrinsics (‘_add‘, ‘_mul‘, ‘_fadd‘, ‘_slt‘, ‘_cmov‘, etc.) that map EXO operations onto Mojo-V instruction semantics.
- Storage-type indirection (‘EXO_UINT64E_STORAGE_TYPE‘, ‘EXO_FP64E_STORAGE_TYPE‘) allowing a build to choose fast, strong, or proof-carrying encrypted memory payload types.

This design gives developers an encrypted-type programming model before a complete LLVM language+backend integration exists.

2) WHAT EXO IS USED FOR

EXO is used to:

1. **Write encrypted algorithms in ordinary C++ syntax**
   - Example benchmarks include bubble sort, GEMM, knapsack, PSI, NTT kernel, etc., each including ‘mojov-exo.h‘ and instantiating EXO encrypted types.

2. **Exercise Mojo-V semantics in bring-up and evaluation**
   - The Mojo-V type tests (‘bringup-bench/mojov-typetests‘) validate arithmetic, comparisons, assignment variants, casts, and conditional moves over EXO types.

3. **Support multiple protection formats from the same source code**
   - By defining storage macros before including EXO, benchmark builds can target fast or strong memory-encryption layouts without rewriting algorithm code.

4. **Enable debug-time plaintext checks in simulation**
   - EXO provides ‘debug_context(...)‘ and ‘.decrypt()‘ support (for test/debug) that decrypt values using SIMON key expansion and a contract signature.

3) HOW EXO WORKS (MECHANICS)

A. TYPE LAYER AND OPERATOR OVERLOADING

EXO introduces encrypted scalar classes:

- ‘inte_t<Bits, IsSigned>‘ for encrypted integers
- ‘fpe_t<Bits>‘ for encrypted floating point (32/64 aliases)

These classes:

- Store encrypted payloads (not plaintext) using selected storage structs.
- Overload arithmetic/logic/comparison operators to preserve C++ ergonomics.
- Return encrypted predicates (rather than plaintext bools), keeping control decisions expressible via data-oblivious mechanisms.

B. INTRINSIC MAPPING THROUGH INLINE ASSEMBLY

Each EXO operator funnels into intrinsics from ‘mojov-intrinsics.h‘.

Examples:

- ‘a + b‘ -> ‘_add(...)‘
- ‘a < b‘ (unsigned) -> ‘_sltu(...)‘
- ‘fp_a + fp_b‘ -> ‘_fadd(...)‘
- ‘cmov(pred, t, f)‘ -> ‘_cmov(...)‘ / ‘_fcmov(...)‘

The intrinsics load encrypted operands, execute arithmetic/relational instructions, and store encrypted results. Logical operators are explicitly implemented without short-circuit behavior to maintain data-oblivious semantics.

C. EXPLICIT DATA-OBLIVIOUS SELECTION MODEL

Instead of relying on plaintext branches over secret data, EXO provides ‘cmov(...)‘ overloads for integer and floating-point cases, enabling branchless secret-dependent selection patterns compatible with Mojo-V expectations.

D. CONFIGURABLE ENCRYPTED MEMORY ABI

By setting:

- ‘EXO_UINT64E_STORAGE_TYPE‘
- ‘EXO_FP64E_STORAGE_TYPE‘

before including ‘mojov-exo.h‘, the same C++ source can bind to:

- ‘mojov_mem_fast_*‘
- ‘mojov_mem_strong_*‘
- ‘mojov_mem_proofcarrying_*‘

This decouples algorithm code from encryption-format representation details.

E. DEBUG DECRYPTION PATH FOR VERIFICATION (NON-PRODUCTION SEMANTIC AID)

For validation runs, EXO can decrypt wrapped values with:

- ‘debug_context(simon_key, contract_sig)‘ initialization
- ‘.decrypt()‘ on encrypted wrappers

This is used by test suites to compare encrypted execution outcomes against expected plaintext behavior.

4) WHY EXO ENABLES PARALLEL LLVM AND LIBRARY/BENCHMARK DEVELOPMENT

EXO is a **staging layer** that removes the immediate dependency on a full LLVM Mojo-V language+codegen port.

PARALLELIZATION EFFECT

- **Compiler/backend team** can work on true first-class LLVM support (IR typing, ABI, codegen, optimization legality, debugging/profiling integration).
- **Library + benchmark teams** can simultaneously develop encrypted algorithms, APIs, and test suites now using EXO wrappers and intrinsics.
- **System team** can validate Mojo-V ISA behavior in Spike and across encryption modes while compiler work is still in progress.

Because EXO is header-first and macro-configurable, most application-facing experimentation can proceed independently from heavyweight compiler milestones.

5) TRADE-OFFS: EXO LIBRARY VS. FULL LLVM PORT

A. BENEFITS OF EXO (SHORT/MEDIUM TERM)

1. **Fast time-to-productivity**
   - Usable immediately with existing C/C++ toolchains and simulator workflows.

2. **Low integration risk early on**
   - Avoids front-loading complex compiler architecture changes before ISA and runtime behavior stabilize.

3. **Great for bring-up and semantic validation**
   - Easy to build focused tests that verify operators, type conversions, and encrypted control primitives.

4. **Flexible encryption-format experimentation**
   - Storage-type macros let teams compare fast/strong/proof-carrying choices from mostly shared code.

B. COSTS/LIMITS OF EXO (COMPARED TO FULL LLVM SUPPORT)

1. **Not a first-class language type system integration**
   - Encrypted semantics live in library wrappers, not native compiler IR types and analyses.

2. **Optimization ceiling**

- Compiler cannot reason globally about encrypted semantics as deeply as with dedicated LLVM IR and passes.

3. **Tooling ergonomics gap**
   - Diagnostics, debugging, and profiling are less seamless than true compiler-native features.

4. **Potential abstraction/ABI friction**
   - Wrapper indirection and explicit intrinsics can create edge-case interoperability concerns with generic libraries and mixed-language boundaries.

5. **Long-term maintenance duplication risk**
   - Some EXO patterns may later be superseded by compiler-native implementations, requiring migration.

C. BENEFITS OF FULL LLVM PORT (LONG TERM)

A complete LLVM port would enable:

- Native encrypted types and operations in the compiler pipeline
- Better legality-aware optimization and scheduling
- Cleaner source-level language integration beyond wrapper idioms
- More robust ecosystem/toolchain support at scale

D. PRACTICAL CONCLUSION

A realistic engineering strategy is:

1. Use EXO now for rapid bring-up, validation, and benchmark acceleration.
2. Continue full LLVM integration in parallel.
3. Gradually transition performance-critical and production-facing paths to compiler-native support as it matures.

In short: **EXO optimizes for velocity and parallel progress; a full LLVM port optimizes for long-term compiler quality, portability, and scale.**

6) BOTTOM LINE

The Mojo-V EXO library is a deliberate bridge between ISA innovation and production compiler maturity. It gives teams a usable encrypted programming model today, proves semantics with real benchmarks, and de-risks the path to eventual first-class LLVM support.